

Resum de l'API basica de sem6000

1. Introduccio

La llibreria sem6000 permet descobrir, connectar i controlar endolls Voltcraft SEM6000 per Bluetooth, a mes de consultar dades de consum i configuracio.

En aquest projecte, la llibreria es troba a

r:\Codis\github2\python3-voltcraft-sem6000. Per fer servir discover() cal tenir Bluetooth actiu i permisos suficients.

2. Patro d'importacio local

El patro recomanat es afegir el repositori local al sys.path abans d'importar el modul principal. Això evita haver d'instal·lar la llibreria globalment.

```
from pathlib import Path
import sys

PROJECT_ROOT = Path(__file__).resolve().parents[1]
LIBRARY_ROOT = PROJECT_ROOT / "github2" / "python3-voltcraft-sem6000"
if str(LIBRARY_ROOT) not in sys.path:
    sys.path.insert(0, str(LIBRARY_ROOT))

from sem6000 import sem6000
```

3. Funcions principals

- Nom: SEM6000.discover(timeout=5, bluetooth_device='hci0')
- Que fa: Escaneja l'entorn Bluetooth i retorna una llista de dispositius compatibles amb el servei del SEM6000.
- Parametres:
- timeout: segons d'espera per detectar dispositius.
- bluetooth_device: interface Bluetooth a utilitzar, habitualment hci0.
- Que retorna: Retorna una llista de diccionaris amb claus com name i address.
- Exemple minim:

```
devices = sem6000.SEM6000.discover()
for device in devices:
    print(device['name'], device['address'])
```

- Errors o observacions importants: Pot requerir permisos elevats segons el sistema. Si no hi ha cap dispositiu visible, la llista pot ser buida.
- Nom: SEM6000(deviceAddr=None, pin=None, bluetooth_device='hci0', timeout=5, debug=False)
- Que fa: Crea el client principal. Si passes deviceAddr, intenta connectar de seguida. Si també passes pin, intenta autoritzar-se.
- Parametres:
- deviceAddr: MAC del dispositiu, per exemple 00:11:22:33:44:55.
- pin: PIN de 4 dígit, per exemple 0000.
- bluetooth_device: interface Bluetooth.

- timeout: temps maxim d'espera per resposta.
- debug: si es True, escriu informacio de depuracio a stderr.
- Que retorna: Retorna una instancia de la classe SEM6000.
- Exemple minim:

```
device = sem6000.SEM6000('00:11:22:33:44:55', pin='0000', debug=True)
```

- Errors o observacions importants: Si la connexio o l'autoritzacio fallen, es llançara una excepcio.
- Nom: connect(device_address)
- Que fa: Connecta el client actual a una MAC concreta.
- Parametres:
- device_address: adreca MAC del dispositiu que vols obrir.
- Que retorna: Retorna el resultat intern de la reconnexion si tot va be.
- Exemple minim:

```
device = sem6000.SEM6000()
device.connect('00:11:22:33:44:55')
```

- Errors o observacions importants: No autoritza per si sol. Despres cal cridar authorize(pin) si cal.
- Nom: authorize(pin)
- Que fa: Envia el PIN al dispositiu connectat i valida l'accés.
- Parametres:
- pin: codi de 4 dígits del dispositiu.
- Que retorna: Retorna un AuthorizedNotification si l'autenticacio te exit.
- Exemple minim:

```
device.authorize('0000')
```

- Errors o observacions importants: Si el PIN es incorrecte o el dispositiu no respon, es llança una excepcio d'autenticacio.
- Nom: request_device_name()
- Que fa: Llegeix el nom configurat al dispositiu remot.
- Parametres:
- Sense parametres.
- Que retorna: Retorna un DeviceNameRequestedNotification amb la propietat device_name.
- Exemple minim:

```
name_info = device.request_device_name()
print(name_info.device_name)
```

- Errors o observacions importants: Serveix per identificar el dispositiu sense canviar-ne l'estat.
- Nom: power_on()

- Que fa: Activa el rele i dona pas de corrent a la sortida de l'endoll.
- Parametres:
- Sense parametres.
- Que retorna: Retorna un PowerSwitchedNotification si el canvi te exit.
- Exemple minim:

```
device.power_on()
```

- Errors o observacions importants: Llana una excepcio si el dispositiu rebutja l'ordre.
- Nom: power_off()
- Que fa: Desactiva el rele i talla el corrent a la sortida.

- Parametres:
- Sense parametres.
- Que retorna: Retorna un PowerSwitchedNotification si el canvi te exit.
- Exemple minim:

```
device.power_off()
```

- Errors o observacions importants: Es l'equivalent de power_on(), pero per apagar l'endoll.
- Nom: request_measurement()
- Que fa: Demana els valors electricos actuals: consum, tensio, corrent, frecuencia i consum total acumulat.
- Parametres:
- Sense parametres.
- Que retorna: Retorna un MeasurementRequestedNotification, no un diccionari.
- Exemple minim:

```
measurement = device.request_measurement()
print(measurement.power_in_milliwatt)
print(measurement.voltage_in_volt)
```

- Errors o observacions importants: Les dades venen en atributs de l'objecte retornat, com power_in_milliwatt o total_consumption_in_kilowatt_hour.
- Nom: request_settings()
- Que fa: Consulta la configuracio activa del dispositiu: limit de potencia, preus i franja reduida.
- Parametres:
- Sense parametres.
- Que retorna: Retorna un SettingsRequestedNotification.
- Exemple minim:

```
settings = device.request_settings()
print(settings.power_limit_in_watt)
```

```
print(settings.normal_price_in_cent)
```

- Errors o observacions importants: Igual que amb `request_measurement()`, el resultat es un objecte amb atributs, no una estructura JSON.
- Nom: `disconnect()`
- Que fa: Tanca la connexió Bluetooth amb el dispositiu actual.
- Parametres:
- Sense parametres.
- Que retorna: Retorna `True` si la desconexió s'executa correctament.
- Exemple mínim:

```
device.disconnect()
```

- Errors o observacions importants: Es recomanable cridar-lo al final per deixar la connexió neta.

4. Exemples d'us

Exemple 1: descobrir dispositius disponibles.

```
devices = sem6000.SEM6000.discover()
for device in devices:
    print(f'{device["name"]}\t{device["address"]}')

```

Exemple 2: connectar-se, autoritzar, llegir nom i una mesura.

```
device = sem6000.SEM6000()
device.connect('00:11:22:33:44:55')
device.authorize('0000')
name_info = device.request_device_name()
measurement = device.request_measurement()
print(name_info.device_name)
print(measurement.power_in_milliwatt)

```

Exemple 3: encendre, apagar i desconnectar.

```
device.power_on()
device.power_off()
device.disconnect()

```

5. Notes practiques i limitacions

- `discover()` necessita accés Bluetooth i pot requerir permisos elevats segons l'entorn.
- `authorize()` falla si el PIN no és correcte o si el dispositiu no respon.
- `request_measurement()` i `request_settings()` retornen objectes de notificació, no diccionaris.
- Aquest patró d'ús assumeix que el dispositiu trobat és compatible amb la llibreria SEM6000.